

Football Tournament Management System

Developer Manual

Technical documentation based on the current source code

Author: Rostand Cedrix Assonfo Demaboi
Institution: Technische Hochschule Georg Agricola, Bochum
Module: Introduction to Database Management Systems
Semester: Summer Term 2026
Version: 1.0

September 10, 2026

Contents

1	Purpose and Scope	2
1.1	Functional Scope	2
2	Technology Stack	2
3	System Architecture	3
3.1	Frontend Layer	3
3.2	Backend Layer	3
3.3	Database Layer	3
4	Repository Structure	4
4.1	Backend Modules	4
4.2	Frontend Modules	5
5	Database Design	5
5.1	Tables	5
5.2	Users Table	5
5.3	Tournaments Table	6
5.4	Teams Table	6
5.5	Players Table	6
5.6	Matches Table	6
5.7	Team–Tournament Association	7
5.8	Indexes	7
6	Backend Implementation	7
6.1	FastAPI Application	7
6.2	Database Access	7
6.3	Tournament Routes	7
6.4	Team Routes	8
6.5	Match Routes	8
6.6	Health Check	8
7	Pydantic Schemas	8
8	Authentication and Authorization	9
8.1	Current Integration Status	9
9	Frontend Architecture	9
9.1	Startup	9
9.2	Connection Dialog	9
9.3	Dashboard	9
9.4	Match Scheduling	10
9.5	Score Update	10
10	Frontend API Client	10
11	Business Logic	10
11.1	Match Validation	10
11.2	Score Validation	10
11.3	Tournament Dates	11
11.4	Standings	11

12 Running the Application	11
12.1 Docker	11
12.2 Backend Without Docker	11
12.3 Frontend	12
13 Database Initialization	12
13.1 SQL Initialization	12
13.2 Seed Data	12
13.3 Python Seed Script	12
14 Development Workflow	12
15 Testing	13
16 Known Issues and Technical Debt	13
16.1 Docker Compose and PostgreSQL	13
16.2 Database Configuration	13
16.3 Authentication Header	14
16.4 Authentication Dependency	14
16.5 Ranking Endpoint	14
16.6 Duplicate API Client Function	14
16.7 Model and Route Naming	14
16.8 Credentials	14
17 How to Extend the Application	14
18 Maintenance Guidelines	15
19 Appendix: Useful Commands	15
19.1 FastAPI Documentation	16

1 Purpose and Scope

This document is the technical developer manual for the Football Tournament Management System. It explains how the current repository is structured, how the backend and frontend communicate, how the SQL database is organized, how to run the application, and where a future developer should modify the code when extending the system.

This manual is based on the source code contained in the submitted project archive. Where the repository contains inconsistencies or unfinished functionality, these points are explicitly identified.

1.1 Functional Scope

The application is intended to manage football competitions, including:

- tournaments;
- teams and coaches;
- players;
- scheduled matches;
- match scores;
- tournament rankings;
- organizer/viewer access concepts.

The SQL schema additionally contains users, player records, and the many-to-many relationship between teams and tournaments.

2 Technology Stack

Component	Current implementation
Backend	Python, FastAPI, Uvicorn
Database access	SQLAlchemy ORM + psycopg2-binary
Database	PostgreSQL
Configuration	python-dotenv / environment variables
Frontend	Python Tkinter
HTTP client	Python requests
Containerization	Docker / Docker Compose
API documentation	FastAPI automatic documentation at /docs
Frontend packaging	pyproject.toml / uv configuration

The backend currently declares:

```
1 fastapi>=0.111.0
2 uvicorn>=0.29.0
3 psycopg2-binary>=2.9.0
4 python-dotenv>=1.0.0
```

The frontend requires Python 3.12 or newer and declares `requests>=2.34.2`.

3 System Architecture

The current application follows a client/server architecture.

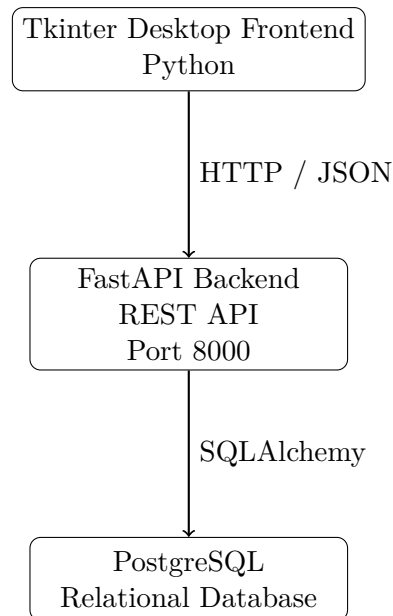


Figure 1: Current application architecture

3.1 Frontend Layer

The frontend is located under:

```
1 frontend/src/football_tournament/
```

It uses Tkinter and provides the connection screen, dashboard, management forms and presentation of tournament data.

3.2 Backend Layer

The backend is located under:

```
1 backend/app/
```

The FastAPI application is created in `main.py`. The current application registers the following routers:

```
1 /tournaments
2 /teams
3 /matches
```

3.3 Database Layer

The SQL database schema is defined in:

```
1 database/init.sql
```

Test data is provided by:

```

1 database/seed.sql
2 backend/seed.py

```

4 Repository Structure

The relevant source structure is:

```

1 football-tournament-manager/
2 |
3 +-- backend/
4 | +-- app/
5 | | +-- models/
6 | | | +-- __init__.py
7 | | +-- routes/
8 | | | +-- tournaments.py
9 | | | +-- teams.py
10 | | | +-- matches.py
11 | | +-- schemas/
12 | | | +-- __init__.py
13 | | +-- database.py
14 | | +-- auth.py
15 | | +-- main.py
16 | +-- requirements.txt
17 | +-- seed.py
18 | +-- Dockerfile
19 | +-- .env
20 |
21 +-- frontend/
22 | +-- src/
23 | +-- football_tournament/
24 | +-- __main__.py
25 | +-- api.py
26 | +-- connection_dialog.py
27 | +-- ui/
28 | +-- dashboard.py
29 | +-- pyproject.toml
30 |
31 +-- database/
32 | +-- init.sql
33 | +-- seed.sql
34 |
35 +-- docker-compose.yml
36 +-- README.md

```

4.1 Backend Modules

Module	Responsibility
app/main.py	FastAPI application and router registration
app/database.py	SQLAlchemy engine, sessions and DB health check
app/auth.py	API token verification helper
app/models/	SQLAlchemy ORM models
app/schemas/	Pydantic schemas
app/routes/tournaments.py	Tournament operations

<code>app/routes/teams.py</code>	Team operations
<code>app/routes/matches.py</code>	Match operations
<code>seed.py</code>	Programmatic test-data insertion

4.2 Frontend Modules

Module	Responsibility
<code>api.py</code>	HTTP requests to the backend
<code>connection_dialog.py</code>	Backend URL and token input
<code>ui/dashboard.py</code>	Dashboard and data display
<code>__main__.py</code>	Main application and management forms

5 Database Design

5.1 Tables

The SQL initialization script creates:

- `users`;
- `tournaments`;
- `teams`;
- `players`;
- `matches`;
- `team_tournaments`.

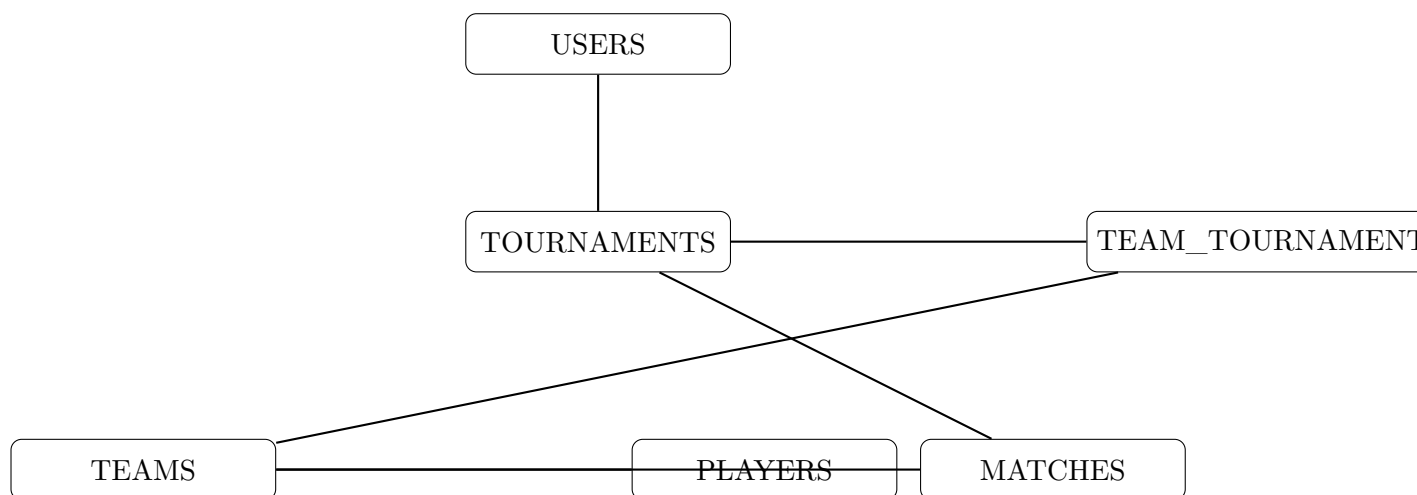


Figure 2: Logical database relationships

5.2 Users Table

Column	Type	Constraint	Meaning
<code>user_id</code>	INTEGER	PK, Identity	User identifier

username	VARCHAR(50)	UNIQUE, NOT NULL	Username
email	VARCHAR(100)	UNIQUE, NOT NULL	Email
password	VARCHAR(255)	NOT NULL	Password/hash field
role	VARCHAR(20)	CHECK	organizer/player/viewer
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT	Creation time

5.3 Tournaments Table

Column	Type	Constraint	Meaning
tournament_id	INTEGER	PK, Identity	Tournament identifier
name	VARCHAR(100)	NOT NULL	Tournament name
start_date	DATE	NOT NULL	Start date
end_date	DATE	NOT NULL	End date
organizer_id	INTEGER	FK, NOT NULL	Organizer user
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT	Creation time

The database enforces:

```
1 CHECK (start_date <= end_date)
```

5.4 Teams Table

Column	Type	Constraint	Meaning
team_id	INTEGER	PK, Identity	Team identifier
name	VARCHAR(100)	UNIQUE, NOT NULL	Team name
coach	VARCHAR(100)	Nullable	Coach name

5.5 Players Table

Column	Type	Constraint	Meaning
player_id	INTEGER	PK, Identity	Player identifier
first_name	VARCHAR(50)	NOT NULL	First name
last_name	VARCHAR(50)	NOT NULL	Last name
position	VARCHAR(30)	Nullable	Playing position
team_id	INTEGER	FK, NOT NULL	Player team

5.6 Matches Table

Column	Type	Constraint	Meaning
match_id	INTEGER	PK, Identity	Match identifier
match_date	DATE	NOT NULL	Match date
home_score	INTEGER	CHECK ≥ 0	Home score
away_score	INTEGER	CHECK ≥ 0	Away score
tournament_id	INTEGER	FK, NOT NULL	Tournament
home_team_id	INTEGER	FK, NOT NULL	Home team
away_team_id	INTEGER	FK, NOT NULL	Away team

created_at	TIMESTAMP WITH TIME ZONE	DEFAULT	Creation time
------------	--------------------------	---------	---------------

The schema also contains:

```
1 CHECK (home_team_id != away_team_id)
```

5.7 Team–Tournament Association

`team_tournaments` implements the many-to-many relationship between teams and tournaments.

Its primary key is:

```
1 PRIMARY KEY (team_id, tournament_id)
```

5.8 Indexes

The SQL initialization script creates indexes on tournament, home-team, away-team, player-team and tournament-organizer foreign-key columns.

6 Backend Implementation

6.1 FastAPI Application

The main FastAPI object is defined in `backend/app/main.py`.

The application exposes:

```
1 GET /
2 GET /health
3
4 /tournaments
5 /teams
6 /matches
```

6.2 Database Access

The database layer uses SQLAlchemy:

```
1 engine = create_engine(DATABASE_URL)
2 SessionLocal = sessionmaker(
3     autocommit=False,
4     autoflush=False,
5     bind=engine
6 )
```

Database sessions are provided to routes using the `get_db()` dependency.

6.3 Tournament Routes

The current implementation contains:

Method	Path	Purpose
GET	/tournaments/	List tournaments
GET	/tournaments/{tournament_id}	Get one tournament

POST	/tournaments/	Create tournament
DELETE	/tournaments/{tournament_id}	Delete tournament

6.4 Team Routes

Method	Path	Purpose
GET	/teams/	List teams
POST	/teams/	Create team
DELETE	/teams/{team_id}	Delete team

6.5 Match Routes

Method	Path	Purpose
GET	/matches/	List matches
POST	/matches/	Schedule a match
PUT	/matches/{match_id}/score	Update score
DELETE	/matches/{match_id}	Delete match

6.6 Health Check

The endpoint:

```
1 GET /health
```

returns the API status and the database connectivity status.

7 Pydantic Schemas

The project defines Pydantic schemas for:

- users;
- tournaments;
- teams;
- players;
- matches;
- team-tournament associations;
- ranking items;
- tournament rankings.

Important classes include:

```
1 TournamentCreate
2 TeamCreate
3 PlayerCreate
4 MatchCreate
5 RankingItem
6 TournamentRanking
```

Not all schemas are currently connected directly to the route functions. Some routes accept dictionaries or local Pydantic classes.

8 Authentication and Authorization

The project contains an authentication helper in `backend/app/auth.py`.

It expects:

```
1 x-api-token: <token>
```

The expected token is read from the environment variable `API_TOKEN`, with a fallback value defined in the source.

The helper returns HTTP 401 for an invalid token.

8.1 Current Integration Status

The helper `verify_token` exists, but the current route registration does not attach it as a FastAPI dependency. Consequently, the routes registered in `main.py` are currently not protected by this helper.

A future developer should integrate the dependency before production deployment.

9 Frontend Architecture

9.1 Startup

The frontend entry point is:

```
1 frontend/src/football_tournament/__main__.py
```

The application opens a connection screen and allows the user to specify the backend URL and an optional token.

9.2 Connection Dialog

The default API URL is:

```
1 http://localhost:8000
```

The current UI labels the token as `X-API-Token`.

9.3 Dashboard

The dashboard is implemented in:

```
1 frontend/src/football_tournament/ui/dashboard.py
```

It contains four tabs:

- Tournaments;
- Standings;
- Teams;
- Matches.

9.4 Match Scheduling

The scheduling form collects:

- tournament;
- match date;
- home team;
- away team;
- referee.

The frontend prevents selecting the same team for both sides.

9.5 Score Update

The score form requires integer score values and calls:

```
1 PUT /matches/{match_id}/score
```

10 Frontend API Client

The API client is located at:

```
1 frontend/src/football_tournament/api.py
```

It provides functions including:

```
1 get_tournaments()
2 create_tournament(...)
3 get_teams()
4 create_team(...)
5 get_matches()
6 create_match_scheduled(...)
7 update_score(...)
8 get_standings(...)
```

The client sends the token through:

```
1 x-api-token
```

11 Business Logic

11.1 Match Validation

Both frontend and database validation prevent a team from playing against itself.

11.2 Score Validation

The database requires non-negative scores:

```
1 CHECK (home_score >= 0)
2 CHECK (away_score >= 0)
```

The frontend additionally checks that entered scores are numeric.

11.3 Tournament Dates

The database requires:

```
1 CHECK (start_date <= end_date)
```

11.4 Standings

The ranking schema contains:

- position;
- team name;
- matches played;
- goals scored;
- goals conceded;
- points.

The current backend route files do not implement the ranking endpoint referenced by the frontend client. The ranking algorithm should therefore be implemented and documented explicitly if standings are intended to be a supported backend feature.

12 Running the Application

12.1 Docker

The current `docker-compose.yml` defines a backend service on port 8000.

Start it with:

```
1 docker compose up --build -d
```

Check services:

```
1 docker compose ps
```

View backend logs:

```
1 docker compose logs backend
```

Stop services:

```
1 docker compose down
```

12.2 Backend Without Docker

From the backend directory:

```
1 pip install -r requirements.txt
2 uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

12.3 Frontend

The frontend project requires Python 3.12 or newer.

A typical development command is:

```
1 cd frontend
2 source .venv/bin/activate
3 python3 -m football_tournament
```

13 Database Initialization

13.1 SQL Initialization

The schema is created by:

```
1 database/init.sql
```

The script drops existing tables and recreates them in dependency order.

13.2 Seed Data

Test data is available in:

```
1 database/seed.sql
```

The SQL seed creates users, one tournament, three teams, team associations, six players and six matches.

13.3 Python Seed Script

The project also contains:

```
1 backend/seed.py
```

This script creates tables using SQLAlchemy metadata and inserts sample records.

The project therefore currently has two seed mechanisms. A future developer should select one authoritative initialization strategy.

14 Development Workflow

For a new feature, the recommended sequence is:

1. Create a Git feature branch.
2. Modify the SQL schema if necessary.
3. Update SQLAlchemy models.
4. Update Pydantic schemas.
5. Implement backend business logic.
6. Add or modify FastAPI routes.
7. Update the frontend API client.
8. Update the Tkinter interface.

9. Add or update tests.
10. Test the database constraints.
11. Update the documentation.

Example:

```
1 git checkout -b feature/new-functionality
2
3 # Make changes
4
5 git add .
6 git commit -m "Add new functionality"
7 git push origin feature/new-functionality
```

15 Testing

The inspected repository does not contain a dedicated automated test suite.

A future test suite should cover:

- database constraints;
- tournament operations;
- team operations;
- match scheduling;
- score updates;
- authentication;
- ranking calculation;
- API error handling.

A score-update test should verify that an existing match receives a valid score, that the API returns success, and that the new values are persisted in the database.

16 Known Issues and Technical Debt

This section records inconsistencies found in the current repository.

16.1 Docker Compose and PostgreSQL

The README describes PostgreSQL as a Docker Compose service, but the current `docker-compose.yml` defines only the backend service. PostgreSQL must therefore either be provided externally or added to Docker Compose.

16.2 Database Configuration

The submitted `backend/.env` defines individual variables such as `DB_HOST`, `DB_NAME`, `DB_USER`, `DB_PASSWORD` and `DB_PORT`.

However, `database.py` reads the single variable `DATABASE_URL`. These configurations are currently not aligned.

16.3 Authentication Header

The authentication helper and frontend use:

```
1 x-api-token
```

while project documentation also refers to X-API-Key. One convention should be selected and applied consistently.

16.4 Authentication Dependency

The `verify_token` helper is not attached to the currently registered routes. Authentication therefore exists in the codebase but is not fully enforced by the current API routing configuration.

16.5 Ranking Endpoint

The frontend contains a function targeting:

```
1 GET /tournaments/{tournament_id}/ranking
```

but the current tournament router does not implement this endpoint.

The dashboard also checks for a method named `get_tournament_ranking`, whereas the API client defines `get_standings`. These interfaces should be unified.

16.6 Duplicate API Client Function

The frontend API module contains multiple definitions of `create_match_scheduled`. Only the last definition is used by Python. Duplicate definitions should be removed.

16.7 Model and Route Naming

The SQLAlchemy Team model defines `coach`, while the team route contains compatibility logic for `coach_name` and `coach`. The application should use one canonical field name.

16.8 Credentials

The project archive contains a backend environment file with database credentials. Secrets should not be committed to source control. Production credentials should be rotated if they have been exposed.

17 How to Extend the Application

A new data-driven feature should normally follow this flow:

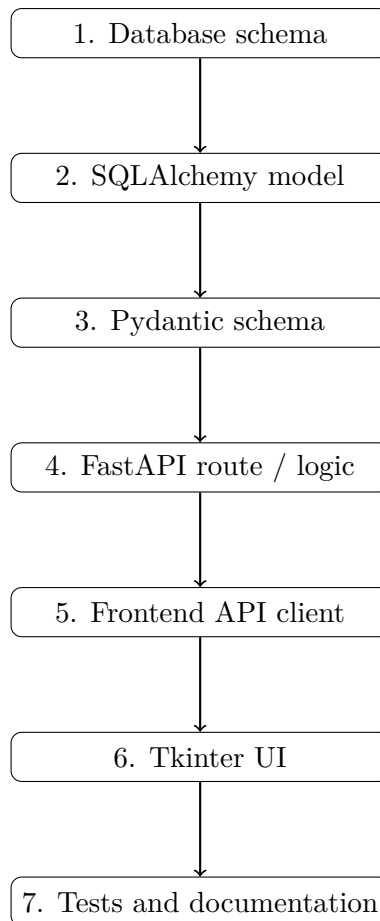


Figure 3: Recommended feature implementation flow

For example, adding player management should update the database, backend models and routes, frontend API client, interface and tests.

18 Maintenance Guidelines

- Keep SQL and SQLAlchemy models synchronized.
- Keep Pydantic schemas synchronized with API contracts.
- Do not commit secrets.
- Use environment variables for deployment-specific settings.
- Keep authentication header names consistent.
- Remove duplicate or obsolete frontend functions.
- Add tests when modifying business logic.
- Update this manual when the architecture changes.

19 Appendix: Useful Commands

Command	Purpose
---------	---------

<code>docker compose up -build -d</code>	Build and start services
<code>docker compose ps</code>	Display service status
<code>docker compose logs backend</code>	Display backend logs
<code>docker compose down</code>	Stop Docker services
<code>uvicorn app.main:app -reload</code>	Run FastAPI in development
<code>python3 -m football_tournament</code>	Start Tkinter frontend

19.1 FastAPI Documentation

When the backend is running, the interactive documentation is normally available at:

1 `http://localhost:8000/docs`

The OpenAPI schema is available at:

1 `http://localhost:8000/openapi.json`